A Performance Analysis of the Condensed K-NN Classifier

Jose Luna jlunabo1@jh.edu
Applied and Computational Mathematics
Johns Hopkins University

Abstract

We explore the performance of k-Nearest Neighbors (k-NN) and its condensed variant (C-kNN) on a range of classification and regression datasets. We evaluate these algorithms using a 5×2 cross-validation framework, comparing them against a null model that predict the majority class or mean target value. Our findings confirm that C-kNN significantly outperforms the naive baseline (by up to 85%) in most cases but is generally variable and dataset-dependent. The condensing algorithm can compress training sets, often by more than 70%, though sometimes at the cost of reduced performance. We also investigate the impact of target skewness on model performance and find no statistically significant effect.
Keywords: Condensed k-Nearest Neighbors, nonparametric learning

1 Problem Statement

The purpose of this project is to explore the performance of nonparametric learning via the Condensed K-Nearest Neighbors algorithm. We will use 5×2 cross-validation to evaluate the performance of this algorithm on 6 datasets from the UC Irvine Machine Learning Repository (3 for classification, 3 for regression). We will then compare to a baseline model, defined as:

- **Null Classifier**: For classification tasks, the Null Classifier always outputs the most common class encountered in the training set.
- **Null Regressor**: For regression tasks, the Null Regressor always outputs the mean of the target values encountered in the training set.

We will test the following hypotheses:

1. The k-NN classifier and predictor will consistently outperform the null models
2. The condensing step of condensed k-NN will significantly reduce the size of the training set, with at most a marginal reduction in performance
3. K-Nearest Neighbors will perform better on datasets whose target values are distributed more evenly. Imbalanced data or outliers can disproportionately influence distance calculations, which will make k-NN less effective.

2 Overview of Datasets

The main characteristics of each dataset are listed below, along with frequency and density histograms for their numerical and categorical target values, respectively. An important

measure(1) for testing our third hypothesis is the *Fisher-Pearson Skewness Coefficient*, defined as:

$$g_1 = \frac{n}{(n-1)(n-2)} \sum \left(\frac{x_i - \bar{x}}{s} \right)^3$$

Where n is the sample size, $\bar{x}$ is the sample mean, and s is the sample standard deviation. A positive value indicates right-skewnese, while a negative value indicates left-skewness, and a value near zero suggests symmetry. This measure was applied to each dataset's target variable to quantify the degree of skew. In particular, we were interested in measuring whether a high degree of skew could impair the performance of distance-based methods like k-nearest neighbors.

Table 1: Summary statistics and target distributions for each classification dataset

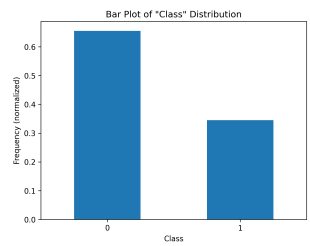
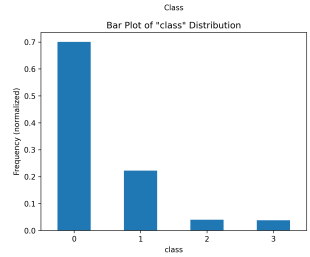
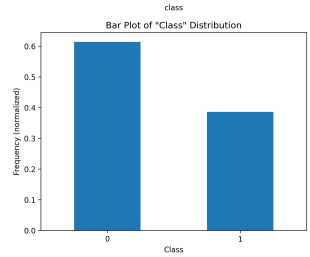
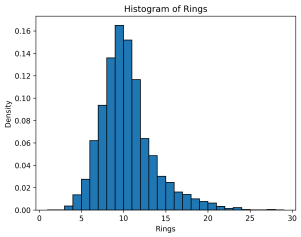
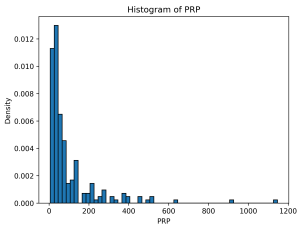
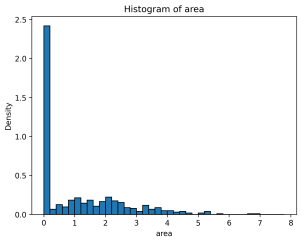
Dataset	**Size (rows)**	**# Features**	**Skewness**	**Target Histogram**
Breast Cancer	699	9	0.655	
Car Evaluation	1728	6	1.980	
Congressional Voting Records	435	16	0.469	

Table 2: Summary statistics and target distributions for each regression dataset

Dataset	**Size (rows)**	**# Features**	**Skewness**	**Target Histogram**
Abalone	4177	8	1.114	
Machine CPU	209	6	3.893	
Forest Fires	517	12	1.218	

Based on Hypothesis (3), we might expect Condensed k-NN to exhibit higher performance on the Breast Cancer Dataset and the Congressional Voting Records dataset than on the Car Evaluation dataset. Similarly, we would expect higher performance on Abalone dataset due to its symmetry. Of course, there are several other factors at play – the density and compactness of each class, and the underlying relationships – but as a crude metric, it is reasonable to assume that Skewness will play a part in performance.

3 Pre-Processing

Each dataset was imported and subjected to the following pre-processing steps:

- **Missing Values**: Missing values were identified using dataset-specific markers and imputed based on feature type. Numerical features were imputed using the mean of the feature, while categorical features were filled with the mode.
- **Feature Encoding**: Categorical features were encoded using one of three schemes:
  - Label encoding for ordinal features: in particular, the "condition" class in the Car dataset
  - One-hot encoding for nominal features; this was applied to almost every feature in the Car and Congress datasets

  - Cyclic encoding for cyclical features such as the day of the week or the month; in particular, the "day" and "month" features of the Forest Fires dataset were interpreted as equally spaced points around a circle (7 points for the day, and 12 points for the month). Then each feature was encoded as a pair of (sin, cos) values in order to preserve a notion of distance.

- **Normalization**: During the tuning and testing phases, each classification or regression run was subject to z-score normalization. The mean and standard deviation were computed for the training set in each case, and applied to both training and test sets.

- **Transforms**: In the Forest Fires dataset, the target (area) feature was heavily skewed towards 0. Following the dataset authors' recommendation, we applied a log transformation of the form $\hat{\mathbf{r}}(x) = \ln(\mathbf{r}(x) + 1)$ where $\mathbf{r}(x)$ is the area associated with example x. All regression predictions and evaluations were performed in this transformed space.

- **Column Filtering**: Unimportant columns such as ID columns and regression prediction columns that could interfere with our predictions were discarded.

4 Experimental Approach

The three models we aimed to compare are:

- **Null Model**: This model is described in the problem statement and serves as a baseline

- **k-Nearest Neighbors**: For a given test point, this model finds the k nearest neighbors in the training set using a given distance metric.

  - For classification tasks, the most frequent class label among the neighbors is returned
  - For regression tasks, the we return a weighted sum of the neighbors' target values, where weights are assigned using the Gaussian Kernel (with a tunable bandwidth parameter γ):
$$K(x, x_q) = \exp\left(-\gamma \|x - x_q\|_2\right)$$

- **Condensed K-Nearest Neighbors**: This model first generates a subset of the training data using by starting with a single point, running k-Nearest Neighbors on each point and iteratively adding incorrectly predicted points to the subset. Note that "incorrect predictions" are defined relative to a tunable tolerance parameter ϵ.

For each dataset, we performed 5×2 Cross-Validation both for hyperparameter tuning and for prediction tasks, and ran the following steps:

- Import the dataset and apply the pre-processing steps outlined above.

- Divide the dataset into 80% - 20% splits, for testing and tuning, respectively.

  - For classification datasets, we stratify by class label to ensure approximately equally distributed classes in each split.

  - For regression tasks, we sample uniformly across the target values.

- Perform hyperparameter tuning
  - For classification tasks, we test a range of p-values for the Minkowski distance (from 1 to 3), and k-values for the nearest neighbors calculations (from 1 to 5).
  - For regression tasks, we test a range of ϵ-values for the condensing step, bandwidths γ for the Gaussian Kernel, and k-values for nearest neighbor calculations (from 1 to 5).
  - For ϵ and γ, we begin with a log-spaced range (0.001, 0.01, 0.1, etc.), then run hyperparameter tuning again to pinpoint the highest performance ranges
  - Then repeat the following five times:
    * Divide the 80% training set into two equally sized partitions (stratified or uniformly sampled), and run the condensed k-NN algorithm using the specified hyperparameters on the each partition, evaluating on the tuning set

- Perform classification / regression. Repeat the following five times:
  - Divide the 80% training set into two equally sized partitions (stratified or uniformly sampled)
  - For each partition:
    * Calculate the mean and standard deviation of this partition, and apply z-score normalization using these statistics to both partitions
    * Run each model to classify or regress the other partition using this partition as a training set
    * Output the performance; for classification, we use the *Error Rate* (percentage of predictions that are incorrect), and for regression we use the *Mean Squared Error*.

- Average the results of each experiment for each candidate model.

5 Results

Table 3: Average MSE and Condensed Set Sizes for Regression Datasets

Dataset	**Average Mean Squared Error**			**Training Set Size**	**Average Condensed Set Length**
	Naive	**kNN**	**C-kNN**		
Abalone	10.4108	5.8333	7.5693	1671	847.9
Machine	26213.0698	9224.7555	9123.6377	84	82.1
Forest Fires	1.9889	2.7841	3.5454	207	120

Table 4: Average Error Rate and Condensed Set Sizes for Classification Datasets

Dataset	**Error Rate**			**Training Set Size**	**Average Condensed Set Length**
	Naive	**kNN**	**C-kNN**		
Breast Cancer	0.3446	0.0522	0.0886	279	29.1
Car	0.3006	0.1461	0.1564	692	200.2
Congress	0.3868	0.0928	0.1129	174	24.5

Table 5: Wilcoxon Signed-Rank Test p-values for Pairwise Comparisons

Dataset	**p: (kNN vs Naive)**	**p: (kNN vs C-kNN)**	**p: (C-kNN vs Naive)**
Abalone	0.00098	0.00098	0.00098
Breast Cancer	0.00098	0.00098	0.00098
Car	0.00098	0.04199	0.00098
Congress	0.00098	0.05273	0.00098
Forest Fires	1.00000	0.03223	1.00000
Machine	0.00098	0.76953	0.00098

Table 6: Hyperparameter values used for each dataset

Dataset	**p (Minkowski Order)**	**k**	ϵ	γ
Breast Cancer	2	1	–	–
Car	1	4	–	–
Congress	2	1	–	–
Abalone	2	3	0.025	0.1
Machine	2	3	0.5	0.5
Forest Fires	2	3	0.01	0.5

6 Discussion

Tables 3 and 4 summarize the average performance across 5×2 cross-validation trials, along with the reduction in training set size due to condensing. Table 5 contains the results of conducting the Wilcoxon Signed-Rank Test(1) on the results of all validation trials. This is a non-parametric statistical test designed to compare groups of paired data. In particular, it provides an alternative to the paired-t test when the data cannot be assumed to come from a normal distribution, as is the case.

For each dataset, we ran the Wilcoxon Signed-Rank test to determine whether there were statistically significant differences between the results of the Null model, the kNN model and C-kNN model (pairwise). Our alternative hypothesis is that the kNN model performs better than the Null model C-kNN model (as is the case for all of our datasets), but the C-kNN model still performs better than the Null model. The table contains the p-values for each test, and is color-coded to highlight the results that are statistically significant ($p < 0.05$).

Table 6 contains the optimal hyperparameters found for each dataset and used for our testing. We now evaluate each of our initial hypotheses:

6.1 Hypothesis 1: *k*-NN Outperforms Null Models

Across all datasets except the Forest Fires dataset, the k-NN algorithm consistently outperformed the naive baseline. This conclusion is supported by Wilcoxon statistical test, which showed in these cases a statistically significant difference between k-NN and the naive model.

For example, in the Breast Cancer dataset, k-NN reduced the average error from 0.3446 to 0.0522, and in the Congress dataset, from 0.3868 to 0.0928. Similar improvements were observed in all other classification datasets except the Forest Fires dataset. For regression tasks, k-NN also showed strong improvements. In the Abalone dataset, the mean squared error dropped from 10.4108 to 5.8333, and in the Machine dataset from 26213.07 to 9224.76.

The one exception is the Forest Fires dataset, where the mean squared error rose from 1.9889 to 2.7831 and the statistical test indicates that there was no significant improvement (in fact, there is a significant degradation in performance). This degradation is likely due

to the extreme skew of the dataset, with most area values at or around 0, even after the log transformation.

Taken together, these results provide strong support for Hypothesis 1 with the caveat that performance is highly dependent on the dataset structure and skew.

6.2 Hypothesis 2: Condensed k-NN Reduces Size with Minimal Accuracy Loss

Condensed k-Nearest Neighbors (C-kNN) was hypothesized to significantly reduce the size of the training set while preserving performance. Our experiments confirm that C-kNN can dramatically reduce the size of the training, though the extent and cost of that reduction varied across datasets.

On average, C-kNN reduced the training set size by:

- **Abalone**: 49.3%
- **Machine**: 2.3%
- **Forest Fires**: 42.0%
- **Breast Cancer**: 89.6%
- **Car**: 71.1%
- **Congress**: 85.9%

In most cases, this reduction came with a significant increase in error. For example, on the Abalone dataset, the mean squared error rose from 5.8333 to 7.5693 after a 49.3% reduction, and in the Forest Fires dataset, MSE increased from 2.7841 to 3.5454.

Some of the performance reductions were more modest, such as on the the Car dataset, where C-kNN increased the error rate from 0.1461 to 0.1564 while shrinking the dataset by over 70%. On the Breast Cancer dataset, performance degradation was similarly mild (0.0522 to 0.0886) despite a nearly 90% reduction.

However, in general, the Wilcoxon signed-rank tests support a statistically significant drop in performance across all datasets except Congress and Machine.

Overall, Hypothesis 2 is only partially confirmed: C-kNN generally achieves major compression of the training set, but the performance degradation can be significant.

6.3 Hypothesis 3: Distribution of Target Values Affects k-NN Performance

To test this hypothesis, we first calculated the percentage performance increase for each dataset between using the Naive model and using k-NN (performance measured as either Error rate or MSE). We then calculated the Pearson Correlation Coefficient between the Skewness Coefficient and this performance difference, as summarized below:

The Pearson Correlation Coefficient is 0.0302, indicating practically no linear relationship between the Skewness coefficient and the performance of k-NN. We conclude that there is little to no support for Hypothesis 3. In the future, it would be worthwhile to explore other metrics for Skewness and do a deeper investigation into the effects of the dataset structure on k-NN.

Dataset	**Skewness**	**Performance Gain (kNN vs. Naive)**
Breast Cancer	0.655	84.87%
Car	1.980	51.39%
Congress	0.469	76.00%
Abalone	1.114	43.97%
Machine	3.893	64.81%
Forest Fires	1.218	−39.98%

Table 7: Skewness and Percentage Performance Improvement of kNN over Naive Baseline

7 Conclusion

In this project, we implemented and evaluated the performance of k-Nearest Neighbors (k-NN) and Condensed k-NN (C-kNN) across six diverse datasets, using both classification and regression tasks to test the following Hypotheses:

1. The k-NN classifier and predictor will consistently outperform the null models
2. The condensing step of condensed k-NN will significantly reduce the size of the training set, with at most a marginal reduction in performance
3. K-Nearest Neighbors will perform better on datasets whose target values are distributed more evenly.

Our results confirm that k-NN consistently outperforms naive baseline models in most cases, achieving statistically significant improvements in prediction accuracy and mean squared error, thus providing support for Hypothesis 1.

As for Hypothesis 2, we found that while C-kNN was able to achieve substantial compression (often exceeding a 70% reduction), this frequently came at the cost of increased error. As a result, our second hypothesis was only partially supported: Condensed k-NN may be a useful tool for data reduction but is often not suitable when accuracy is critical.

Finally, we investigated the relationship between skewness in the target variable and k-NN performance. Despite observing some qualitative trends, statistical analysis showed no significant correlation, offering no support for our third hypothesis.

References

[1] Snedecor, G. W., & Cochran, W. G. (1989). *Statistical Methods* (8th ed.). Iowa State University Press.